Class 15: Feature Engineering — Cleaning, Curation, Selection, Construction

Data-driven Systems Engineering

Agenda

- Why feature engineering remains necessary once every variable is numerically encoded.
- Data cleaning: missing values, duplicates, and column-specific imputation.
- Data curation: documenting and validating datasets for reuse.
- Feature selection: filter, wrapper, and embedded methods.
- Feature construction from domain knowledge, for tabular data and data streams.
- Leakage and availability checks before deployment.

Why feature engineering remains necessary after encoding

- Encoding (Class 14) makes a variable numerical; it does not establish whether the variable is correct, reliable, or informative.
- Real datasets contain missing values, duplicates, and inconsistent types, which must be resolved before an encoding is meaningful.
- Not every available variable should become a feature: some are redundant, some carry only noise, and some induce leakage.
- Informative features are frequently absent from the raw data and must be constructed from domain knowledge.

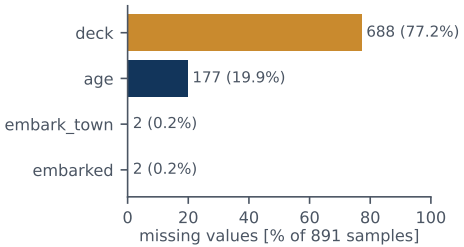
Four related but distinct tasks

Cleaning	Treatment of missing values, duplicates, invalid values, and type errors.
Curation	Organization, documentation, and validation of datasets so that they can be reused and trusted.
Selection	Choice of a subset $S \subseteq \{1, \ldots, d\}$ of the existing features.
Construction	Definition of new features from domain logic, time, aggregation, or combinations of existing ones.

Data cleaning

- Cleaning begins with an inventory of missing values, duplicates, and data types, before any modeling decision.
- The treatment of missing values depends on the meaning of each column and on the missingness mechanism; a single global rule is rarely appropriate.
- For categorical variables, an explicit `Unknown` category often preserves more information than imputation, since missingness itself may be informative.
- Cleaning decisions must be documented, because they change the meaning of the data for every downstream user.

Cleaning example: column-specific imputation



Titanic data used in `notebooks/Class4_DataEngineering.ipynb`, $n = 891$.

Column	Policy	Rationale
`deck`	explicit `Unknown`	77% missing: imputation would invent most values; missingness may be informative
`age`	median (28.0)	numerical, moderate missingness; the median is robust to outliers
`embarked`	mode (`S`)	two missing values: negligible effect
`embark_town`	explicit `Unknown`	duplicates `embarked` (see the χ^2 ranking)

The dataset also contains 107 duplicated rows; without a passenger identifier, they cannot be distinguished from distinct passengers with identical attributes, so they were kept.

Data curation

- Curation makes a cleaned dataset reusable by analysts other than those who cleaned it.
- A data dictionary records the meaning, unit, and valid range of every column.
- Dataset versioning, or at least a recorded extraction date, allows later experiments to be compared on equal terms.
- Validation rules (schema and range checks) detect upstream problems before they reach the modeling stage.

Repository example

The specification of the fraud case in `documents/FraudDetectionAsaService.pdf` acts as a curated data description: it documents the dataset and the workflow explicitly instead of leaving them implicit in a notebook.

Curation example: a data dictionary for the pasteurization sensors

Variable	Unit	Meaning	Observed range	Validation rule
`T`	°C	Product temperature; set point 72 during Hold	3.95 – 72.24	$0 \le T \le 80$
`pH`	—	Acidity of the milk	6.60 – 6.69	$6.4 \le \mathrm{pH} \le 6.9$
`Kappa`	mS/cm	Electrical conductivity	2.91 – 5.27	$\kappa > 0$
`Mu`	cP	Viscosity; decreases with temperature	1.42 – 2.45	$\eta > 0$
`Tau`	relative	Turbidity	0.14 – 1.17	$\tau \ge 0$
`Q_in, Q_out`	n.s.	Inflow and outflow; 1.5 when active, 0 otherwise	0.00 – 1.70	$Q \ge 0$
`P`	n.s.	Pressure	0.81 – 1.62	$P > 0$
`dTdt`	°C/s	Derived: $(T_t - T_{t-1})/\Delta t$	-5.48 – 3.61	—

Observed ranges over 100 batches simulated with `synth_sensors.py`; n.s.: unit not specified in the simulator; validation rules are proposed examples. Such a table, versioned with the data, is the core of a datasheet for the dataset.

Feature selection: filter, wrapper, embedded

Given d candidate features, selection seeks a subset $S \subseteq \{1, \dots, d\}$ that maximizes a criterion $J(S)$. The three families differ in how J is defined.

Filter	J is a model-independent statistic computed for each feature, e.g. correlation, mutual information, or the χ^2 statistic.
Wrapper	$J(S)$ is the validation performance of a model trained on S; subsets are explored by a search strategy (e.g. forward or backward selection).
Embedded	Selection occurs during training, e.g. the least absolute shrinkage and selection operator (Lasso), $\min_{\beta} \|\mathbf{y} - X\boldsymbol{\beta}\|_2^2 + \lambda\|\boldsymbol{\beta}\|_1$, which sets some coefficients exactly to zero.

Filter example: the χ^2 test of independence

Observed counts O (expected E under independence), `sex` $\times$ `survived`:

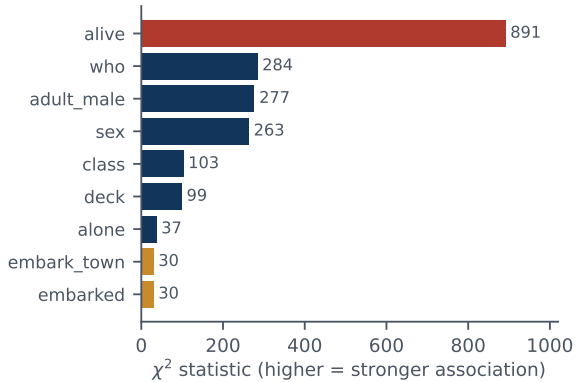
	died	survived	total
female	81 (193.5)	233 (120.5)	314
male	468 (355.5)	109 (221.5)	577
total	549	342	891

$$\chi^2 = \sum_{\text{cells}} \frac{(O-E)^2}{E} = 263.1$$

with $E = (\text{row total} \times \text{column total})/n$.

- If `sex` and `survived` were independent, each cell would contain approximately its expected count E.
- The larger χ^2, the further the observed table is from independence, i.e. the stronger the association with the target ($p < 10^{-58}$ here, with 1 degree of freedom).
- It is a filter criterion: computed per feature, before and independently of any model.
- **Caution:** `sklearn.feature_selection.chi2` treats feature values as non-negative counts. Applied to label-encoded nominal variables, it does not compute this test, and its score depends on the arbitrary codes (92.7 instead of 263.1 for `sex`).

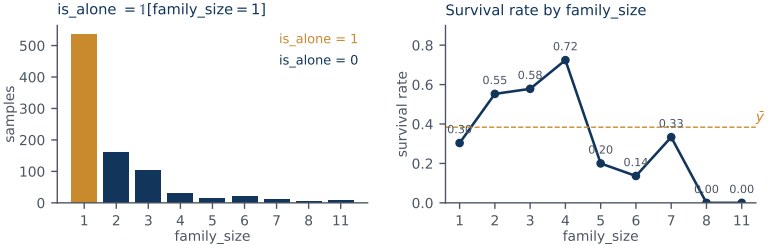
Filter result: χ^2 ranking of categorical features



- **alive** attains the maximum $\chi^2 = n = 891$: it restates the target (`yes`/`no`). A filter ranks a leaking feature first; it must be removed by inspection, not by score.
- **embarked** and **embark_town** have identical scores: they encode the same information, so one of them is redundant.
- `who`, `adult_male`, and `sex` rank high together because they overlap strongly; a univariate filter cannot detect this redundancy.

χ^2 test of independence on the contingency table of each feature with `survived`; missing values treated as an explicit category.

Selection example: is `is_alone` redundant?



- **Informationally redundant:** `is_alone` is a deterministic function of `family_size`; it adds no information, which would justify removing it.
- **Not redundant for a linear model:** survival is non-monotonic in `family_size`, so its linear correlation with `survived` is only 0.02, whereas that of `is_alone` is -0.20. The indicator acts as a non-linear transformation that a linear model cannot build by itself.
- Redundancy is therefore relative to the model family: a decision tree can recover `is_alone` with a single split; logistic regression cannot. The notebook keeps it, and the test accuracy of its logistic regression rises from 0.804 to 0.810 (one additional correct prediction out of 179).

Feature construction from domain knowledge

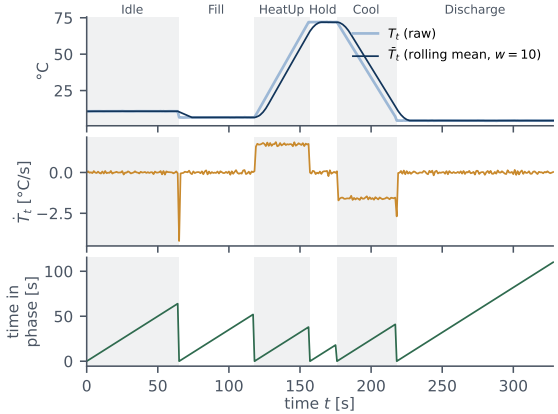
- Informative features usually originate from an understanding of the problem rather than from generic mathematical transformations.
- Aggregations (counts, sums), ratios, and simple domain rules often outperform automated feature generation.
- Each constructed feature should be checked for redundancy with the variables from which it was derived.
- The rationale for every constructed feature should be stated explicitly, in one sentence.

Construction example: domain rules

Feature	Definition	Scale	Rationale
`family_size`	$\texttt{sibsp} + \texttt{parch} + 1$	ratio	Passengers travelling together share their fate; survival rate rises from 0.30 (alone) to 0.72 (four members), then falls
`is_alone`	$\mathbf{1}[\texttt{family_size} = 1]$	binary	Isolates the largest and least favoured group (537 passengers)
`title`	`man`→Mr, `woman`→Mrs, `child`→Master	nominal	One-to-one relabelling of `who`: adds no information (identical χ^2)

- From `notebooks/Class4_DataEngineering.ipynb`: each constructed feature is a short, explainable rule. A useful construction combines or transforms information; a bijective relabelling, such as `title`, only renames it.
- In the fraud case, behavioural features (rolling transaction count, deviation from recent spending) would require a card identifier, which the anonymized dataset does not provide.

Feature construction for streams and sensors



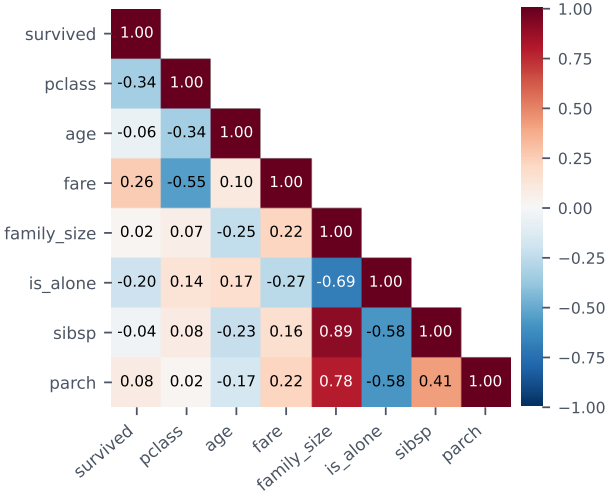
One batch generated by `pasteurization/synth_sensors.py`.

Features computed from the raw temperature T_t:

- **Rolling mean:** $\bar{T}_t = \frac{1}{w}\sum_{i=0}^{w-1} T_{t-i}$; smooths noise and indicates stability, at the cost of a lag of about $w/2$ samples.
- **Rate of change:** $\dot{T}_t = (T_t - T_{t-1})/\Delta t$; separates HeatUp ($> 0$) from Cool ($< 0$), which the raw value cannot.
- **Time in phase:** seconds since the last change of state; a state-aware feature that distinguishes a slow heating from a normal one.

All three use only past observations, so they can be computed online, one sample at a time, exactly as in production.

Engineered features change the dependence structure



- Pearson correlations after cleaning (age imputed with the median).
- `family_size` is strongly correlated with its components `sibsp` (0.89) and `parch` (0.78): keeping all three adds collinearity without new information.
- `is_alone` has a larger linear association with `survived` (-0.20) than `family_size` (0.02), as discussed in the selection example.
- Pearson correlation measures only linear dependence; a near-zero value does not imply that a feature is uninformative.

Leakage and availability checks

- Is the feature available at prediction time, or does it use information from the future?
- Does it encode a post-decision outcome, such as the result of a manual review?
- Is it computed identically in production and in the notebook?
- Does it depend on joins or upstream systems that may be unavailable at inference time?

The complete taxonomy of leakage (target, preprocessing, temporal, group, duplicate, model selection) was presented in Class 13.

Repository connections

Repo activity

- `notebooks/Class4_DataEngineering.ipynb` covers cleaning, χ^2 selection, and domain-driven construction end to end.
- `pasteurization/synth_sensors.py` provides the constructed feature for time series data (`dTdt`).
- `streamlit/modeling.py` saves the fitted scaler together with the model, illustrating that preprocessing decisions must be persisted with the model rather than applied once in a notebook.

Papers and materials

- Guyon, I., Elisseeff, A. (2003). An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3, 1157–1182.
- Kohavi, R., John, G. H. (1997). Wrappers for feature subset selection. *Artificial Intelligence*, 97(1–2), 273–324.
- Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B*, 58(1), 267–288.
- Little, R. J. A., Rubin, D. B. (2019). *Statistical Analysis with Missing Data*, 3rd ed. Wiley.
- Gebru, T., Morgenstern, J., Vecchione, B., Wortman Vaughan, J., Wallach, H., Daumé III, H., Crawford, K. (2021). Datasheets for datasets. *Communications of the ACM*, 64(12), 86–92.
- Domingos, P. (2012). A few useful things to know about machine learning. *Communications of the ACM*, 55(10), 78–87.
- Zheng, A., Casari, A. (2018). *Feature Engineering for Machine Learning*. O'Reilly.
- scikit-learn User Guide: feature selection; pipelines and composite estimators.

Take-away message

Features are design decisions:

clean according to column semantics, select with evidence, construct from domain knowledge, and use only what is available at prediction time.

Thank you for your attention.

Next: Class 16 — machine learning modeling, experiment tracking, evaluation, and decision-making.